



ИНСТИТУТ ЗА МАТЕМАТИКУ И ИНФОРМАТИКУ
ПРИРОДНО-МАТЕМАТИЧКИ ФАКУЛТЕТ
УНИВЕРЗИТЕТ У КРАГУЈЕВЦУ

ЗАВРШНИ РАД

**СИСТЕМ ЗА ДЕТЕКЦИЈУ БЕЖИЧНИХ
УРЕЂАЈА ТОКОМ ИСПИТНИХ СЕСИЈА
ПРИМЕНОМ *ESP32* ПЛАТФОРМЕ**

Ментор
др Ана Капларевић-Малишић

Студент
Андрија Костовић, 51/2021

Крагујевац, септембар 2026.

Садржај

Предговор	4
1. Увод	5
1.1. Мотивација и циљеви	5
1.2. Полазно стање и обим имплементације	6
2. Преглед технологија	7
2.1. Клијентска страна	7
2.2. Серверска страна	7
2.3. <i>ESP32</i> и сензорски слој	8
2.4. <i>MQTT</i> , <i>PostgreSQL</i> и <i>Docker</i> окружење	8
3. Архитектура система	9
3.1. Преглед компоненти	9
3.2. Слојевита организација серверске апликације	9
3.3. Трофазни процес обраде опажања	10
3.4. Шема базе података	11
3.5. <i>Backend REST API</i>	12
3.6. Серверски модели и уговор поруке	13
3.7. <i>Frontend</i> странице и компоненте	13
3.8. Клијентски сервиси	14
3.9. Позадински процес за пријем мерења	14
4. Детекција, локализација и процена ризика	15
4.1. <i>RSSI</i> и модел удаљености	15
4.2. Распоред сензора и избор опажања	15
4.3. Пондерисана процена положаја	16
4.4. Показатељ поузданости и Калманово изглађивање	17
4.5. Израчунавање показатеља ризика	17
5. Управљање сесијама и додатне функционалности	19
5.1. Животни циклус испитне сесије	19
5.2. Листа дозвољених уређаја	19
5.3. Аларми и потврда прегледа	19
5.4. Извештаји и <i>CSV</i> извоз	20
6. Обрада грешака и стабилност система	21
6.1. Валидација улазних података	21
6.2. <i>HTTP</i> грешке и дневници	21
6.3. Поновљена испорука и прекиди комуникације	21
7. Безбедност система	22
7.1. Аутентикација и ауторизација	22
7.2. Псеудонимизација идентификатора	22
7.3. Ревизиони записи и задржавање опажања	22
7.4. Границе развојне конфигурације	23
8. Перформансе и оптимизације	24
8.1. Процена обима података	24
8.2. Потенцијална уска грла	24
8.3. Освежавање приказа и мерење кашњења	24
9. Тестирање	25
9.1. Јединични тестови	25

9.2. Тестови контролера и интеграциона провера	25
9.3. Симулатор сензора	25
9.4. План провере на физичким сензорима	26
10. Одржавање и унапређење система	27
10.1. Конфигурација и испорука	27
10.2. Приоритети софтверског унапређења	27
10.3. Хардверска и алгоритамска проширења	28
10.4. Корисничке и организационе допуне	28
11. Закључак	29
Литература	30
Кратка биографија кандидата	31

Предговор

Овај рад представља технички опис система за детекцију бежичних уређаја и праћење њихове активности током испитних сесија. Систем је замишљен као подршка наставнику у уочавању појава које захтевају додатну проверу, уз обраду метаподатака о примљеном сигналу. Основу решења чине сензорски чворови засновани на платформи *ESP32*, *MQTT* комуникациони канал, серверска апликација развијена на платформи *.NET* и кориснички интерфејс реализован помоћу *Angular* оквира.

У доступној верзији пројекта реализовани су пријем и чување опажања, управљање испитним сесијама, израчунавање показатеља ризика, формирање упозорења и процена положаја уређаја. За локализацију су примењени модел слабљења сигнала, линеаризовани поступак пондерисаних најмањих квадрата и Калманово изглађивање координата. Кориснику су доступни контролна табла, мапа учионице, листа уређаја, преглед аларма, листа дозвољених уређаја и збирни извештаји.

Посебан део решења представља софтверски симулатор сензора. Он генерише поруке у истом формату који је предвиђен за *ESP32* чворове, чиме омогућава испитивање серверске обраде и приказа без непосредног повезивања физичких уређаја. Изворни код фирмвера и резултати мерења у стварној учионици нису приложени у анализираном репозиторијуму. Због тога се у раду раздвајају реализоване софтверске функционалности, предвиђена хардверска поставка и поступци експерименталне провере које је потребно спровести.

Рад је организован тако да се најпре представе проблем и употребљене технологије, а затим архитектура, модел података и ток обраде. Посебна поглавља посвећена су локализацији, управљању сесијама, стабилности, безбедности, перформансама и тестирању. Завршни део разматра могућности даљег развоја и домет предложеног решења. Технички опис заснован је на стању доступног изворног кода прегледаног 10. септембра 2026. године [1].

1. Увод

Употреба мобилних телефона, паметних сатова и других бежичних уређаја поставља додатне захтеве пред организацију испита у физичком простору. Наставник има ограничен увид у активност уређаја који нису непосредно видљиви. Истовремено, само присуство радио-сигнала не говори довољно о разлогу његовог настанка: сигнал може потицати од дозвољене опреме, суседне просторије или уређаја који се аутоматски оглашава.

Предмет овог рада је развој софтверске основе за прикупљање и анализу таквих опажања. Уместо обраде садржаја комуникације, систем користи идентификатор извора, ознаку сензора, јачину примљеног сигнала, врсту сигнала и време детекције. Обрада је организована у оквиру испитне сесије, што омогућава повезивање догађаја са одређеним простором и временским интервалом.

Добијени резултат има саветодавни карактер. Упозорење означава да је скуп улазних података задовољно дефинисана правила, а не да је доказана недозвољена радња. Таква поставка одређује и начин приказа: наставнику се пружају подаци за проверу, док се одлука о њиховом значењу задржава у оквиру непосредног надзора испита.

1.1. Мотивација и циљеви

Основна мотивација је повезивање више извора мерења у јединствен информациони систем. Појединачно *RSSI* читавање, односно показатељ јачине примљеног сигнала, има ограничену вредност. Када се повеже са временом, положајем сензора, претходним опажањима и списком дозвољене опреме, оно постаје погодније за тумачење.

Циљ имплементације је да обезбеди непрекидан ток података од извора мерења до наставничког интерфејса. Тај ток обухвата пријем порука, проверу основних ограничења, псеудонимизацију идентификатора, чување опажања и израчунавање показатеља ризика. Додатни циљ је приближно позиционирање уређаја на мапи учионице на основу читавања више сензора.

Са становишта софтверског инжењерства, пројекат треба да покаже како се издвајају доменска правила, приступ бази и комуникациони механизми. Важан захтев је могућност развоја без сталне доступности хардвера. Због тога је симулатор део пројекта, а формат његових порука усклађен је са уговором за будуће сензорске чворове.

Основни циљеви и њихова реализација.

- Прикупљање опажања. Реализација: *MQTT Worker* и *REST* пријем. Начин провере: Симулиране поруке и преглед базе.
- Праћење испита. Реализација: Сесије *planned*, *active* и *completed*. Начин провере: Провера дозвољених прелаза.
- Уочавање ризичних догађаја. Реализација: Правила бодовања и праг 70. Начин провере: Познати улази и очекивани резултати.
- Приближна локализација. Реализација: *WLS* и изглађивање координата. Начин провере: Синтетички и будући теренски подаци.
- Приказ наставнику. Реализација: *Angular* странице и *SignalR*. Начин провере: Провера приказа и освежавања.

- Ограничено чување идентификатора. Реализација: *SHA-256* са *salt* и *pepper* вредностима. Начин провере: Преглед начина уписа у базу.

1.2. Полазно стање и обим имплементације

Полазне спецификације предвиђају три *ESP32* сензора, детекцију *Wi-Fi* и *Bluetooth* активности и процену локације уређаја. У њима се разматрају и сложенији обрасци, као што су међусобна близина уређаја и боравак ван очекиване зоне. При описивању реализације потребно је проверити који су од тих захтева заиста преточени у код.

Доступна имплементација садржи детерминистички сервис за процену ризика. Он користи јачину сигнала, информацију да ли је уређај нов у бази, припадност листи дозвољених уређаја и текстуалну ознаку врсте сигнала. У том сервису нису реализовани анализа путања, мерење растојања између парова уређаја нити модел машинског учења.

Такође, подршка за вредност *bluetooth_pairing* у поруци показује да сервер уме да обради тако означен догађај. Она сама по себи не потврђује да је физички сензор способан да поуздано препозна сваки покушај упаривања. За то је потребна посебна имплементација и експериментална провера на одабраном хардверу.

У наставку се као реализовано описује оно што је присутно у изворном коду. Планиране могућности разматрају се као унапређења. На тај начин документ представља проверљив опис пројекта и основу за његову наредну фазу.

2. Преглед технологија

Технологије су изабране тако да раздвоје прикупљање мерења, пословну обраду и кориснички приказ. Сензорском делу потребан је једноставан формат порука, серверском делу поуздано чување и обрада, док клијентска апликација треба да омогући преглед података уз што мање ручног освежавања.

2.1. Клијентска страна

Клијентска апликација реализована је у *Angular* оквиру. У датотеци *package.json* наведене су зависности из гране 20.3, *TypeScript* из гране 5.9 и *RxJS* из гране 7.8. Ови подаци описују верзије декларисане у пројекту, а не препоруку за избор најновијег окружења [1].

Странице су организоване као самосталне компоненте које обједињују шаблон, стилове и понашање. Стање се у већем броју компоненти чува кроз *signal*, док *computed* служи за извођење зависних вредности. *Angular* сигнали прате промене вредности које компоненте користе при приказу [5]. У пројекту се тај механизам примењује, на пример, за број активних уређаја и број непотврђених аларма.

HTTP комуникација је издвојена у заједничке сервисе. Компонента иницира операцију, сервис формира захтев, а добијени резултат мења њено стање. Оваква организација спречава понављање путања и транспортне логике у свакој страници.

Руте за контролну таблу, мапу, уређаје, аларме, дозвољене уређаје и извештаје читавају се преко *loadComponent*. Страница за пријаву доступна је посебно, док су остале странице смештене у заједнички распоред и заштићене клијентском провером пријаве. Серверска ауторизација остаје неопходна без обзира на ту проверу.

2.2. Серверска страна

Серверска апликација написана је у језику *C#* и циља *.NET 9. ASP.NET Core API* садржи контролере за пријаву, сесије, пријем опажања, уређаје, аларме, дозвољене уређаје и извештаје. Контролери позивају апликационе сервисе, док се приступ бази обавља преко инфраструктурног слоја [1].

Entity Framework Core повезује доменске моделе са *PostgreSQL* базом. Промене структуре воде се кроз миграције, које описују прелаз између верзија модела. Овај механизам омогућава постепено усклађивање базе са изменама апликације [7]. У пројекту су издвојене почетна миграција, додавање спољног идентификатора опажања, ревизионог записа и трајне аутентикације.

MQTT поруке прима засебан *.NET* процес *Worker*. Он користи *MQTTnet* библиотеку и позива исти апликациони интерфејс за пријем података који користи и *API*. Тиме се доменска обрада не везује искључиво за један транспортни протокол.

За обавештења је употребљен *SignalR*. Он омогућава серверу да шаље податке повезаним клијентима, уз подршку за различите транспортне механизме [6]. У овој апликацији служи за догађаје о уређајима и алармима, док мапа положаје преузима периодичним *HTTP* захтевима.

2.3. ESP32 и сензорски слој

Предвиђени хардвер заснива се на класичној *ESP32* серији, за коју произвођач наводи *Wi-Fi* у опсегу 2,4 GHz и *Bluetooth BR/EDR* и *BLE* подршку [2]. При избору конкретне плоче потребно је проверити тачан модел: назив породице *ESP32* сам по себи није довољна спецификација радио-могућности.

У предложеној поставци сензори имају познате координате и објављују опажања са својим идентификатором. За *Wi-Fi* је предвиђено пасивно ослушкивање, док *BLE* део треба заснивати на пријему огласних пакета. *Bluetooth Classic inquiry* није исто што и потпуно пасивно слушање, па се тај поступак не може без додатног објашњења подвести под пасивну детекцију целог система.

Wi-Fi и *Bluetooth* деле радио-ресурс, а документација описује временску поделу и ограничења коегзистенције [3]. Зато интервали скенирања из почетне спецификације представљају пројектни предлог. Њихова изводљивост и утицај на *MQTT* пренос морају се проверити у фирмверу и стварној поставци.

2.4. MQTT, PostgreSQL и Docker окружење

MQTT користи модел објављивања и претплате: извор шаље поруку на тему, а посредник је доставља заинтересованим претплатницима. Пројекат користи *Mosquitto* као посредник и теме облика *sensors/{sensorId}/rssi*. *Worker* се претплаћује на *sensors/+rssi*. Ниво *QoS 1* дозвољава поновљену испоруку, због чега је потребно препознавање већ обрађених догађаја [4].

PostgreSQL чува податке о сесијама, уређајима и опажањима. *Docker Compose* описује заједничко покретање базе, *MQTT* посредника, *API*-ја, *Worker* процеса и клијентске апликације. Симулатор је издвојен у профил *tools*, што омогућава његово покретање само када је потребан за демонстрацију или проверу.

Технолошки састав доступне верзије.

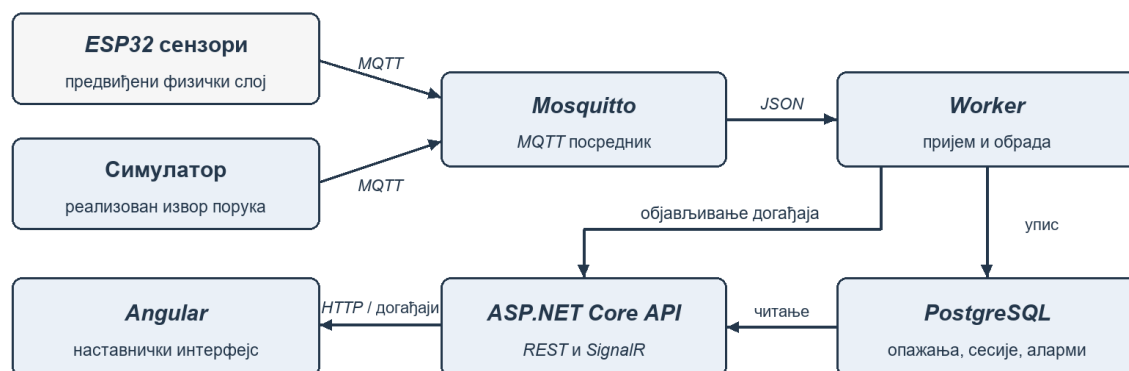
- Кориснички интерфејс. Технологија: *Angular*, *TypeScript*, *SCSS*. Улога: Приказ и управљање.
- *HTTP* сервер. Технологија: *ASP.NET Core*, *.NET 9*. Улога: *REST* операције и аутентикација.
- Позадински процес. Технологија: *.NET Worker*, *MQTTnet*. Улога: Претплата и пријем порука.
- Пословна логика. Технологија: *C#* сервиси. Улога: Сесије, ризик и локализација.
- Складиште. Технологија: *PostgreSQL 16*, *EF Core*. Улога: Трајно чување података.
- Обавештења. Технологија: *SignalR*. Улога: Догађаји ка клијентима.
- Посредник. Технологија: *Eclipse Mosquitto 2*. Улога: *MQTT* размена.
- Симулатор. Технологија: *Node.js*. Улога: Синтетичка сензорска опажања.
- Покретање. Технологија: *Docker Compose*. Улога: Повезивање компоненти.

3. Архитектура система

Архитектура система раздваја извор мерења од његове обраде и приказа. Исти формат опажања може потицати од симулатора или будућег физичког сензора. Серверски део затим примењује заједничка правила, а резултати се чувају у бази и стављају на располагање клијентима.

3.1. Преглед компоненти

Систем чине сензорски слој, *MQTT* посредник, *Worker*, апликациони сервиси, база података и *API* са клијентским интерфејсом. Слика 1 приказује границе тих компоненти. *API* и *Worker* су засебни извршни процеси, али деле доменску логику и инфраструктуру за приступ истој бази.



API и Worker користе заједничке слојеве Application, Domain и Infrastructure.

Слика 1: Компоненте система и главни правци размене података.

Ова организација ближа је слојевитој апликацији са издвојеним процесом за пријем него скупу независних микросервиса. Не постоје засебна складишта и независни доменски модели за сваку функционалност. Раздвајање процеса омогућава да *MQTT* пријем има сопствени животни циклус, док се правила обраде одржавају на једном месту.

API прихвата корисничке захтеве и објављује *SignalR* чвориште. *Worker* користи сопствени токен са улогом *Worker* да би преко тог чворишта проследио обавештења. Клијент тако не мора да зна који је процес првобитно обрадио догађај.

3.2. Слојевита организација серверске апликације

Domain садржи ентитете и основна правила њиховог настанка и промене. На пример, *DeviceObservation* проверава опсег *RSSI* вредности, а *ExamSession* ограничава прелазе између стања. Та правила не зависе од *HTTP* захтева или *MQTT* библиотеке.

Application дефинише случајеве употребе и интерфејсе према спољним зависностима. У њему се налазе сервиси за сесије, ризик, локализацију, листу дозвољених уређаја и извештаје. Интерфејси репозиторијума омогућавају да пословна логика приступа подацима без непосредног познавања начина њиховог чувања.

Infrastructure реализује те интерфејсе помоћу *EF Core* контекста и репозиторијума. У овом слоју је и *LoggingRssiProcessingPipeline*, који координира упис опажања, проверу листе дозвољених уређаја, позив сервиса за ризик и објављивање догађаја.

Назив класе не треба тумачити као ограничење на логовање, јер она обавља главни ток пријема.

Api и *Worker* представљају улазне тачке. Они региструју зависности и повезују транспорт са апликационом логиком. *Dependency injection* омогућава да сервис добије одговарајући репозиторијум или објављивач догађаја без ручног конструисања целог стабла зависности.

Одговорности серверских слојева.

- *Domain*. Примери: *Device*, *ExamSession*, *Alert*. Одговорност: Доменски подаци и инваријанте.
- *Application*. Примери: *RiskScoringService*, *PositionQueryService*. Одговорност: Случајеви употребе и алгоритми.
- *Infrastructure*. Примери: *AppDbContext*, репозиторијуми. Одговорност: Чување и инфраструктурна обрада.
- *Api*. Примери: Контролери, *MonitoringHub*. Одговорност: *HTTP* и *SignalR* приступ.
- *Worker*. Примери: *MqttWorker*. Одговорност: *MQTT* пријем и позадински рад.

3.3. Трофазни процес обраде опажања

Прва фаза обухвата пријем и претварање *JSON* садржаја у улазни модел. *Worker* чита *payload*, преузима идентификатор уређаја и сензора, врсту сигнала и *RSSI*, а затим формира *RssiIngressMessage*. Ако временска ознака није прослеђена, користи се текуће серверско време.

Друга фаза обухвата трајну обраду. Најпре се проверава да ли *eventId* већ постоји. Затим се идентификатор уређаја хешира и тражи одговарајући запис. За нови уређај креира се ентитет, док се постојећем ажурира време последњег опажања. Ново опажање повезује се са уређајем и, ако је наведена, сесијом.

Трећа фаза обухвата процену ризика и обавештавање. Сервис проверава листу дозвољених уређаја и израчунава резултат. После чувања опажања објављује се *deviceUpdated*. Када је резултат најмање 70 и није пронађен недавни аларм за исти уређај и сесију, формира се *Alert* и шаље *alertCreated*.

Локализација је засебан пут читања. *PositionQueryService* рачуна положаје када клијент затражи */positions*. Због тога се не може тврдити да је свако ново опажање већ у пријемном процесу претворено у положај и трајно сачувано. У тренутном моделу нема посебне табеле историје процењених координата.



Слика 2: Ток пријема опажања и издвојени поступак процене положаја.

3.4. Шема базе података

Централни објекти података су уређај и његова опажања. Запис у табели *devices* представља уређај под псеудонимизованим идентификатором, док *device_observations* садржи појединачна мерења. Тако се непроменљиви идентитет записа раздваја од великог броја временски означених читавања.

Табела *exam_sessions* чува назив, просторију, време почетка и завршетка и статус сесије. Табела *alerts* садржи догађаје који су прешли праг ризика, а *whitelist_entries* дозвољене уређаје са интервалом важења. Помоћне табеле *users*, *refresh_tokens* и *audit_logs* подржавају приступ и евиденцију операција.

Основне табеле и одабрана поља.

- *devices*. Одабрана поља: *Id*, *hash_id*, *type*, *first_seen_at*, *last_seen_at*. Намена: Препознати идентификатори.
- *device_observations*. Одабрана поља: *Id*, *device_id*, *sensor_id*, *session_id*, *rsi*, *captured_at*, *external_id*. Намена: Појединачна опажања.
- *exam_sessions*. Одабрана поља: *Id*, *name*, *room_id*, *starts_at*, *ends_at*, *status*. Намена: Контекст испита.
- *alerts*. Одабрана поља: *Id*, *device_id*, *session_id*, *risk_score*, *reason*, *acknowledged_at*. Намена: Упозорења и потврде.
- *whitelist_entries*. Одабрана поља: *Id*, *session_id*, *student_ref*, *device_hash*, *valid_from*, *valid_to*. Намена: Дозволе у оквиру сесије.
- *users*. Одабрана поља: *Id*, *username*, *password_hash*, *Role*. Намена: Кориснички налози.
- *refresh_tokens*. Одабрана поља: *Id*, *user_id*, *token_hash*, *expires_at*, *revoked_at*. Намена: Обнова приступа.
- *audit_logs*. Одабрана поља: *Id*, *actor*, *action*, *resource*, *status_code*, *created_at*. Намена: Евиденција измена.

3.4.1. Односи међу подацима

Један уређај може имати више опажања и више аларма. Једној сесији логички припада више опажања, аларма и дозвола. Ови односи видљиви су кроз идентификаторе које сервиси користе у упитима.

У доступној *EF* конфигурацији нису експлицитно дефинисане релације преко *HasOne* и *HasMany*. *SessionId* се у више ентитета чува као текст, док је примарни идентификатор сесије *Guid*. Стога логичке везе описане у раду не треба изједначити са потпуно дефинисаним страним кључевима у бази. Увођење тих ограничења представља могуће побољшање интегритета података.

Конфигурација сензора налази се у подешавањима апликације, а распоред учионица у клијентском коду. Не постоји засебан трајни модел свих сензора и њихових положаја по просторији. Тај избор поједностављује прототип, али ограничава управљање већим бројем различитих простора.

3.4.2. Индекси и миграције

За *hash_id* уређаја и *external_id* опажања дефинисани су јединствени индекси. Они подржавају проналажење уређаја и контролу понављања догађаја. Додатни индекси постоје над временом опажања и комбинацијом сесије и времена, што одговара честим начинима претраге.

Индекси могу убрзати проналажење редова, али уносе трошак при изменама података [8]. Зато њихов ефекат у овом систему треба проценити на стварном обиму опажања. Само постојање индекса не гарантује да су упити који учитавају целу историју сесије довољно ефикасни.

Миграције се налазе у *Infrastructure/Persistence/Migrations*. *API* при покретању позива *Database.Migrate*, а затим иницијализује развојни налог. За локално окружење то поједностављује прво покретање. У контролисаном окружењу примену миграција треба уклопити у поступак испоруке и израде резервних копија.

3.5. Backend REST API

REST API организован је по ресурсима. Контролери за функционалности наставника захтевају улогу *Professor*. Пријава, обнова и опозив токена налазе се у *AuthController*, док су стање и спремност сервиса доступни кроз посебне *health* путање.

Одабране реализоване *API* операције.

- *POST /api/auth/login*. Намена: Пријава и издавање токена.
- *POST /api/auth/refresh*. Намена: Обнова приступа.
- *GET /api/sessions*. Намена: Преглед сесија.
- *POST /api/sessions*. Намена: Креирање планиране сесије.
- *POST /api/sessions/start-for-room*. Намена: Креирање и покретање за просторију.
- *POST /api/sessions/{id}/start*. Намена: Покретање постојеће сесије.
- *POST /api/sessions/{id}/stop*. Намена: Завршетак сесије.
- *GET /api/sessions/{id}/positions*. Намена: Процењени положаји.
- *GET /api/sessions/{id}/alerts*. Намена: Аларми сесије.
- *POST /api/alerts/{id}/acknowledge*. Намена: Потврда прегледа аларма.

- *GET /api/devices/active*. Намена: Уређаји у временском прозору.
- *POST /api/ingestion/observation*. Намена: Пријем једног опажања.
- *POST /api/ingestion/batch*. Намена: Пријем од 1 до 500 опажања.
- *GET /api/reports/session/{sessionId}*. Намена: Збирни извештај.
- *POST /api/sessions/{id}/whitelist*. Намена: Додавање дозволе.
- *DELETE /api/sessions/{id}/whitelist/{entryId}*. Намена: Уклањање дозволе.

Пријем преко *HTTP*-а враћа статус 202 након позива обраде. Избор тог статуса не значи да је у коду реализован трајни ред за накнадну обраду: контролер чека завршетак позваног сервиса. Пакетни захтев обрађује ставке редом и није представљен као једна атомична трансакција за све ставке.

Путања за активне уређаје користи временски прозор, подразумевано десет минута, без параметра сесије. Насупрот томе, положаји и аларми могу се захтевати за одређену сесију. Та разлика је важна када се тумаче бројеви у различитим деловима интерфејса.

3.6. Серверски модели и уговор поруке

RssiIngressMessage представља улаз у заједничку обраду. Одвојени *Worker* и *HTTP* модели преводе транспортне податке у овај запис. На тај начин се назив поља *timestamp* у *MQTT* поруци и *CapturedAt* у *HTTP* захтеву могу свести на исту семантику времена опажања.

Следећи пример приказује синтетичку *MQTT* поруку. Вредности су илустративне и не представљају забележено мерење:

```
{
  "eventId": "S1-demo-0001",
  "deviceIdentifier": "02:00:00:00:00:01",
  "sensorId": "S1",
  "sessionId": "11111111-1111-1111-1111-111111111111",
  "signalType": "wifi",
  "rssi": -62,
  "timestamp": "2026-09-10T10:00:00Z"
}
```

eventId је предвиђен за препознавање поновљене испоруке. *deviceIdentifier* је улазни идентификатор који се пре трајног чувања претвара у хеш. *sensorId* одређује извор мерења, а *sessionId* контекст испита. *signalType* разликује *wifi*, *ble*, *bluetooth* и *bluetooth_pairing* у уговору поруке. *rssi* је број између -120 и 0 dBm.

DeviceObservation проверава да идентификатор уређаја није празан, да сензор има ознаку, да је *RSSI* у дозвољеном опсегу и да време није више од пет минута у будућности. Међутим, та класа не проверава да ли *sensorId* постоји у конфигурацији нити да ли *sessionId* означава активну сесију. То су разлике између документованог уговора и тренутног нивоа његове примене.

3.7. Frontend странице и компоненте

Кориснички интерфејс подељен је према задацима наставника. Контролна табла служи као почетни преглед, мапа приказује положаје, а странице уређаја и аларма омогућавају детаљнији увид. Посебне странице намењене су дозволама и извештајима. Заједнички *MainLayout* обезбеђује навигацију.

Мапа учионице користи координатни систем ширине осам и дубине шест јединица. Функције *left* и *top* претварају координате у процентуални положај *HTML* ознака. Положаји су визуелно ограничени тако да ознаке остану унутар приказа. То је својство интерфејса и не представља додатну физичку проверу локације.

Компонента мапе преузима положаје на сваке две секунде. Засебан часовник се ажурира сваке секунде и користи се за приказ старости података. Ознаке млађе од осам секунди сматрају се активним; затим им се смањује непрозирност, а после 45 секунди се више не приказују ако су и даље присутне у локалном скупу.

Сервер истовремено примењује свој прозор од 15 секунди у односу на најновије опажање сесије. Зато ознака може нестати раније ако наредни одговор сервера више не садржи тај уређај. Опис постепеног нестајања треба посматрати заједно са начином избора серверских података.

3.8. Клијентски сервис

Api сервис дефинише типизоване операције и моделе одговора. *AuthService* управља пријавом и локално сачуваним токенима, док *HTTP* пресретац додаје приступни токен одговарајућим захтевима. *Realtime* сервис одржава *SignalR* везу и прима обавештења.

Realtime чува листе уређаја и аларма у сигналимa. За *deviceUpdated* постојећи уређај ажурира се по хеш идентификатору, а нови се додаје у листу. Догађај *alertAcknowledged* мења време потврде одговарајућег аларма. Тако се приказ може променити без поновног учитавања целе странице.

Сервис предвиђа аутоматско поновно повезивање после прекида већ успостављене везе. Почетни неуспех повезивања поставља статус *disconnected*. Поновно успостављање везе не подразумева и поновну испоруку свих догађаја насталих током прекида, па је за потпуно усклађивање потребно поново преузети актуелно стање преко *API*-ја.

3.9. Позадински процес за пријем мерења

MqttWorker је изведен из *BackgroundService*. При успостављању везе претплаћује се на конфигурисану тему, а за сваку пристиглу поруку отвара опсег зависности и преузима *IRssiIngestionService*. Тиме појединачна обрада добија одговарајући животни век објекта за приступ бази.

Ако веза није успостављена, процес поново покушава повезивање. После неуспеха прави паузу од пет секунди. Изузеци при обради појединачне поруке бележе се у дневнику, што спречава да један неисправан *JSON* непосредно прекине целу петљу рада.

Не постоји посебан ред неисправних порука, нити трајни механизам поновног објављивања обавештења после уписа у базу. То значи да континуитет процеса и потпуност обраде нису исто својство. За јаче гаранције потребно је повезати чување пословног догађаја и његово накнадно слање.

4. Детекција, локализација и процена ризика

Алгоритамски део повезује три различита питања: да ли је опажање примљено, где се извор сигнала приближно налази и да ли улазни подаци задовољавају правила за упозорење. Ова питања користе заједничке податке, али дају резултате различитог значења и поузданости.

4.1. *RSSI* и модел удаљености

LocalizationService користи модел у коме се удаљеност процењује из разлике између референтног *RSSI* на једном метру и примљеног *RSSI*. У коду су фиксно постављени референтни ниво -45 dBm и експонент слабљења 2.7 [1]. Примењени израз је:

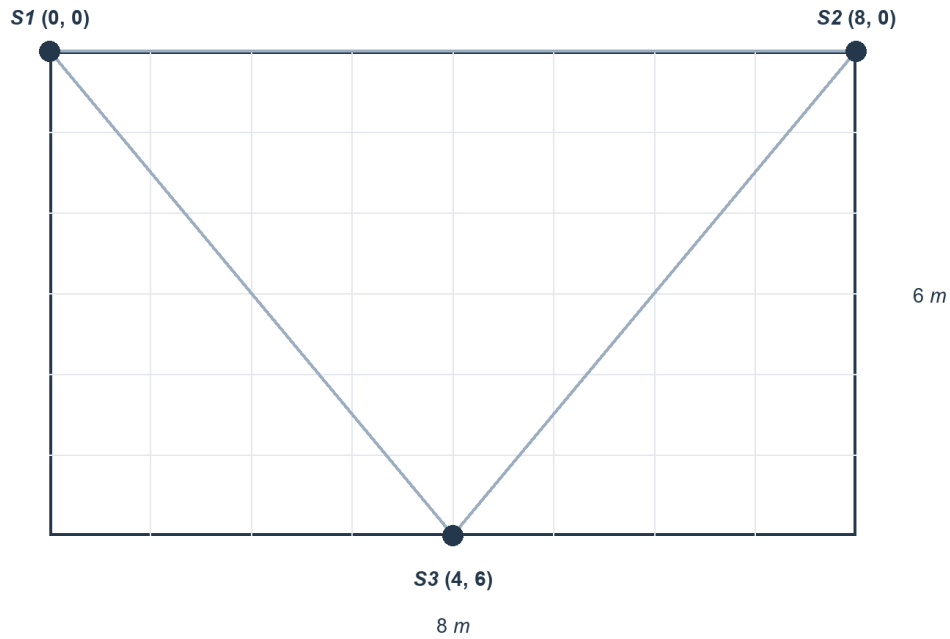
$$d = 10^{\frac{(A - \text{RSSI})}{(10 \cdot n)}} \\ A = -45\text{ dBm}; n = 2,7$$

Величина d представља процењену удаљеност у метрима ако су координате и референтно мерење дефинисани у том систему јединица. За *RSSI* од -45 dBm израз даје један метар. За -72 dBm добија се десет метара. То су рачунски примери из задатих параметара, а не резултати теренског мерења.

Модел није директно мерење растојања. Иста јачина сигнала може настати у различитим условима, а фиксни параметри не описују све положаје и оријентације уређаја. Зато је пре процене практичне тачности неопходно прикупити контролна мерења у просторији у којој ће систем радити.

4.2. Распоред сензора и избор опажања

Подразумевана конфигурација поставља $S1$ у тачку (0, 0), $S2$ у (8, 0) и $S3$ у (4, 6). Такав распоред формира троугао, што обезбеђује неколинеарну геометрију за процену две координате. Слика 3 представља шему из конфигурације, без тврдње да су сензори физички постављени.



Шематски приказ конфигурације; без експерименталних мерења.

Слика 3: Предвиђени положаји сензора у моделу просторије 8×6 m.

PositionQueryService најпре учитава опажања сесије. Од најновијег времена у том скупу одузима конфигурисани прозор, подразумевано 15 секунди. Затим задржава читавања познатих сензора и за сваки уређај узима последње опажање по сензору.

Прозор није везан непосредно за текуће серверско време. Ако нових порука нема, последња група може и даље бити кандидат за процену, а клијент њену старост процењује засебно. Осим тога, читавања различитих сензора у истом прозору не морају бити истовремена. Код покретног уређаја та разлика може утицати на доследност улазних удаљености.

4.3. Пондерисана процена положаја

Када су доступна најмање три читавања, сервис претвара *RSSI* у удаљености и израчунава тежину сваког читавања. Јачи сигнали добијају већу тежину према изразу $w = 10^{((RSSI + 100) / 20)}$. Најјаче читавање служи као референтно.

Одузимањем једначине референтне кружнице од осталих добија се линеаран систем по x и y . Код сабира коефицијенте нормалних једначина и решава систем димензије два. Ако је детерминанта сувише мала, примењује се резервни поступак пондерисаног тежишта.

$$2(x_i - x_0)x + 2(y_i - y_0)y = d_0^2 - d_i^2 - x_0^2 + x_i^2 - y_0^2 + y_i^2$$

$$(A^T W A) p = A^T W b, \quad p = [x, y]^T$$

Код тачно три сензора добијају се две независне линеарне једначине. У недегенерисаном случају њихово тачно решење не добија додатну робусност самим увођењем тежина; њихова улога је израженија када постоји више једначина него непознатих. Зато број сензора и распоред треба разматрати заједно са избором алгоритма.

Добијене координате ограничавају се на најмању и највећу координату конфигурираних сензора по свакој оси. То је правоугаоно ограничење. Оно не доказује да је извор сигнала унутар учионице, нити представља проверу припадности троуглу сензора.

За мање од три читавања користи се пондерисано тежиште положаја сензора. Тај резултат омогућава приближан приказ и када подаци нису потпуни. Међутим, један сензор не пружа јединствено решење положаја у равни, па такав приказ треба тумачити као резервну процену.

4.4. Показатељ поузданости и Калманово изглађивање

После процене положаја рачуна се просечна апсолутна разлика између геометријске удаљености до сензора и удаљености из *RSSI* модела. Показатељ *confidence* добија се као $1 / (1 + residual)$, ограничен на интервал од нуле до један. Код пондерисаног тежишта користи се поједностављена вредност зависна од броја читавања.

Ова величина је хеуристички показатељ усклађености модела и улаза. Она није статистички калибрисана вероватноћа да се уређај налази у одређеној тачки. Висока вредност може постојати и када су улазне удаљености систематски погрешне.

KalmanPositionSmoother затим засебно изглађује координате x и y . За сваки уређај чува процену и грешку у меморији процеса. Процесни шум износи 0,01, мерни шум 2,0, а почетна грешка по координати 1 [1]. Један корак изглађивања дат је следећим изразима:

$$\begin{aligned}P^- &= P + Q \\K &= P^- / (P^- + R) \\x^{\wedge} &= x^{\wedge} + K \cdot (z - x^{\wedge}) \\P &= (1 - K) \cdot P^-\end{aligned}$$

Овде је z нова координата, K појачање, а P процењена грешка. Исти поступак примењује се на другу координату. Ово је једноставан скаларни филтер по оси, без стања брзине и без временског корака у моделу кретања.

Изглађивање се позива при упиту за положаје. Зато поновљени упити могу поново обрадити исту групу мерења и утицати на стање филтера. Стање је везано за идентификатор уређаја, а не за пар који чине уређај и сесија. За строжу временску интерпретацију потребно је везати ажурирање за ново мерење и дефинисати ресетовање филтера.

4.5. Израчунавање показатеља ризика

RiskScoringService реализује правило бодовања чији је резултат цео број од 0 до 100. Основни допринос зависи од јачине сигнала. Нови уређај добија додатних 20 поена, дозвола смањује резултат за 30, ознака упаривања додаје 30, а *Bluetooth* или *BLE* ознака још пет.

$$\begin{aligned}P &= clamp(round(100 - |RSSI|), 0, 100) \\R &= clamp(P + U - W + B + T, 0, 100) \\U &= 20 \text{ за нови уређај; иначе } 0 \\W &= 30 \text{ за дозвољени уређај; иначе } 0 \\B &= 5 \text{ за Bluetooth/BLE; иначе } 0 \\T &= 30 \text{ ако signalType садржи „pair“; иначе } 0\end{aligned}$$

Израз „непознат уређај“ у овом коду значи да запис са датим хешом није постојао пре обраде поруке. То није исто што и уређај који нема дозволу у текућој сесији.

Уређај откривен на претходном испиту може бити познат бази, а да нема дозволу на следећем испиту.

Рачунски примери резултата постојећег правила.

- *RSSI: -80 dBm*. Врста сигнала: *wifi*. Нов уређај: да. Дозвољен уређај: не. Резултат: 40. Праг 70: није достигнут.
- *RSSI: -45 dBm*. Врста сигнала: *wifi*. Нов уређај: да. Дозвољен уређај: не. Резултат: 75. Праг 70: достигнут.
- *RSSI: -45 dBm*. Врста сигнала: *wifi*. Нов уређај: не. Дозвољен уређај: не. Резултат: 55. Праг 70: није достигнут.
- *RSSI: -45 dBm*. Врста сигнала: *wifi*. Нов уређај: не. Дозвољен уређај: да. Резултат: 25. Праг 70: није достигнут.
- *RSSI: -50 dBm*. Врста сигнала: *bluetooth_pairing*. Нов уређај: не. Дозвољен уређај: не. Резултат: 85. Праг 70: достигнут.
- *RSSI: -20 dBm*. Врста сигнала: *bluetooth_pairing*. Нов уређај: не. Дозвољен уређај: да. Резултат: 85. Праг 70: достигнут.

Примери показују да дозвола умањује ризик, али не искључује аутоматски могућност аларма. Такође, након првог уписа истог уређаја нестаје допринос за нови уређај. Ове особине су последица садашње формуле и треба их имати у виду при подешавању правила.

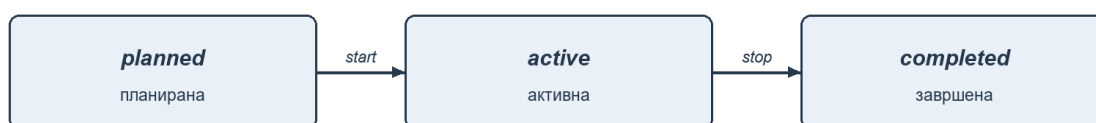
Аларм се креира за резултат најмање 70, ако за исти уређај и сесију није пронађен аларм у претходна два минута. Та провера смањује понављање обавештења. Она није исто што и статистичка анализа аномалија, а истовремена обрада више порука захтева додатну контролу конкуренције ако се тражи строга забрана дуплирања аларма.

5. Управљање сесијама и додатне функционалности

Сесија повезује простор, време и резултате надзора. Њено издвајање омогућава да се опажања и дозволе посматрају у контексту одређеног испита, уместо као један неограничени ток активности.

5.1. Животни циклус испитне сесије

Нова сесија има статус *planned*. Покретањем прелази у *active*, а заустављањем у *completed*. Доменски модел не дозвољава поновно покретање завршене сесије. Апликациони сервис пре покретања проверава да ли у истој просторији већ постоји активна сесија.



Завршена сесија се не покреће поново.

Слика 4: Дозвољени основни прелази стања испитне сесије.

Операција *start-for-room* обједињује креирање и покретање. То одговара току рада у коме наставник изабере учионицу и одмах започне читавање. Промена стања се чува и објављује као *sessionStateChanged*.

Провера активне сесије тренутно је апликациона. Два истовремена захтева могу захтевати јачу заштиту на нивоу базе или трансакције. Слично томе, пријемна обрада сама не одбија свако опажање које се позива на завршену сесију. Управљање животним циклусом и валидацију долазних порука зато треба посматрати као повезане, али одвојене механизме.

5.2. Листа дозвољених уређаја

WhitelistEntry повезује сесију, референцу корисника или студента, хеш уређаја и временски интервал важења. Наставник може да дода уређај који је дозвољен у оквиру одређеног испита, као што је опрема потребна за организацију наставе.

При додавању се користи исти сервис хеширања као при пријему опажања. То је неопходно да би се исти улазни идентификатор препознао у оба поступка. Дозвола утиче на бодовање, а при преузимању положаја информација о дозвољености може бити укључена у одговор за мапу.

Референца *student_ref* служи организацији дозвола. Она не доказује да је уређај у тренутку сваког опажања код конкретне особе. Такво повезивање зависи од поступка регистрације и провере који се спроводи ван алгорита за *RSSI*.

5.3. Аларми и потврда прегледа

Аларм садржи уређај, сесију, резултат ризика, разлог и време настанка. Поље *acknowledged_at* бележи да је аларм прегледан. Потврђивање не мења првобитни резултат нити брише запис, већ издваја обрађена обавештења од оних која тек треба проверити.

Тренутни разлог аларма је општа порука о прекорачењу прага. За практичнију употребу било би корисно приказати и појединачне доприносе резултату. Наставник би тако могао да разликује јак сигнал новог уређаја од догађаја означеног као *Bluetooth* упаривање.

5.4. Извештаји и CSV извоз

ReportingService формира збирни извештај са бројем опажања, бројем аларма, бројем различитих уређаја и временом генерисања. Клијентска страница преузима тај одговор за изабрану сесију и омогућава CSV извоз.

CSV се формира у прегледачу на основу тренутно приказаног извештаја. Доступна реализација не садржи сложен *PDF* извештај, графиконе трендова нити трајно архивирање путања кретања. Збирни бројеви одражавају податке који су у том тренутку присутни у бази.

Брисање старих опажања утиче на касније генерисање извештаја. После примене политике задржавања број опажања и различитих уређаја може се смањити, док аларми могу остати сачувани. Ако је потребан непроменљив завршни извештај, потребно је посебно сачувати његов снимак пре брисања детаљних података.

6. Обрада грешака и стабилност система

Систем прима податке из више извора и зато мора да разликује неисправан улаз, недозвољену промену стања и инфраструктурни отказ. Јединствен начин приказивања грешака поједностављује клијентски код, али не замењује контролу поузданости целог тока.

6.1. Валидација улазних података

HTTP пријем користи атрибуте за обавезна поља, ограничење *RSSI* опсега и дозвољене вредности *signalType*. Пакетни пријем ограничава број елемената на 500. Доменски модел затим примењује основна правила независно од транспорта.

MQTT пут не пролази кроз *HTTP* атрибуте. Он се ослања на десеријализацију и доменске провере. Зато присуство правила у *HTTP* моделу не значи да су иста правила у потпуности примењена на *MQTT* поруку. На пример, непозната непразна ознака сензора може бити сачувана, али се касније не користи за позиционирање ако није у конфигурацији.

Време више од пет минута у будућности доменски се одбија, док само постојање старе временске ознаке није разлог за одбијање у приказаном коду. За потпуну контролу потребно је дефинисати старост дозвољених догађаја и понашање при накнадној испоруци.

6.2. *HTTP* грешке и дневници

ApiExceptionHandler преводи *ArgumentException* у *HTTP* 400, *InvalidOperationException* у 409, а остале изузетке у 500. За неочекивану грешку клијент добија општу поруку, док се детаљи бележе у серверском дневнику. Поједини контролери додатно враћају 404 када тражени ресурс није пронађен.

У пријемном процесу вредности које се уносе у дневник делимично се нормализују заменом знакова новог реда. За идентификатор уређаја користи се хеш. Овај избор смањује непотребно појављивање сирових идентификатора у редовним порукама о обради.

6.3. Поновљена испорука и прекиди комуникације

Провера *external_id* служи да поновљена порука са истим непразним идентификатором не створи ново опажање. Јединствени индекс представља додатно ограничење при упису. Ако *eventId* није наведен, овај поступак не обезбеђује препознавање дупликата.

Упис опажања, слање *deviceUpdated* и креирање аларма нису једна атомична операција. Ако објављивање догађаја не успе после првог чувања, опажање може остати у бази, док наредни корак са алармом није завршен. Поновљени *eventId* тада може зауставити поновну обраду на самом почетку. За поузданију испоруку потребан је трајни запис догађаја за накнадно слање, уз идемпотентну обраду.

Пућања */health* потврђује да *API* одговара, а */health/ready* покушава повезивање са базом. Те провере не потврђују да сви *ESP32* сензори шаљу мерења или да је *MQTT* ток потпун. Надзор целог система захтева и време последње поруке по сензору.

7. Безбедност система

Безбедност обухвата приступ интерфејсу, интегритет догађаја и ограничење података који се чувају. У пројекту су реализовани аутентикација, улоге, хеширање идентификатора и део ревизионе евиденције. Развојна конфигурација ипак није исто што и завршена поставка за употребу у установи.

7.1. Аутентикација и ауторизација

TokenService проверава корисничко име и хеш лозинке и издаје приступни *JWT* и токен за обнову. *JWT* садржи идентификатор, име и улогу корисника, а потписује се *HMAC SHA-256* механизмом. Подразумевано трајање приступног токена је 60 минута, док је рок токена за обнову седам дана [1].

У бази се чува хеш токена за обнову. При успешној обнови стари токен се опозива, а издаје нови пар. *API* проверава издаваоца, публику, потпис и важење приступног токена. Операције наставника захтевају улогу *Professor*, а објављивање преко *PublishFromWorker* улогу *Worker*.

AuthService на клијенту чува токене у *localStorage*. Одјава брише локално стање, али у приказаној функцији *logout* не позива серверски опозив. Зато брисање токена из прегледача не треба описивати као аутоматско поништавање свих раније издатих токена.

7.2. Псеудонимизација идентификатора

Sha256DeviceHashingService уклања спољне размаке, претвара идентификатор у мала слова и рачуна *SHA-256* над комбинацијом *salt*, идентификатора и *pepper* вредности. У трајни запис уређаја уписује се добијени хеш.

```
normalized = trim(identifier).toLowerCase()
hash = SHA256(salt + ":" + normalized + ":" + pepper)
```

Исти улаз уз иста подешавања даје исти хеш, што омогућава повезивање опажања. Управо та повезивост значи да поступак треба описати као псеудонимизацију. Он не обезбеђује да се записи више ни на који начин не могу довести у везу са уређајем или корисником.

Сирови идентификатор постоји у улазној поруци пре хеширања. Заштита базе зато не решава сама за себе заштиту *MQTT* преноса. Такође, промена *pepper* вредности мења добијене хешеве, па ротација тог параметра захтева планиран поступак ако треба очувати постојеће дозволе.

7.3. Ревизиони записи и задржавање опажања

AuditMiddleware бележи одређене аутентификоване захтеве који мењају стање: *POST*, *PUT*, *PATCH* и *DELETE*, уз изузимање путања за аутентикацију. Запис садржи извршиоца, метод, путању ресурса, статус одговора и време. Обични *GET* прегледи нису обухваћени овим механизмом.

ObservationRetentionWorker периодично проналази завршене сесије чији је крај старији од конфигурисаног рока, подразумевано 24 часа, и брише њихова опажања. Провера се понавља на сат времена. То није тренутно брисање тачно 24 часа после завршетка, а прекид рада процеса може додатно одложити поступак.

Овај механизам не брише аутоматски уређаје, аларме, дозволе и ревизионе записе, нити опажања без сесије. Политику задржавања свих категорија података потребно је дефинисати одвојено. Само постојање хеширања и временског брисања једне табеле није довољно за тврдњу о потпуној организационој или правној усклађености.

7.4. Границе развојне конфигурације

Приложена *Mosquitto* конфигурација дозвољава анонимно повезивање на порт 1883, а *Worker* подешава *TCP* везу без приказане *TLS* конфигурације. *Docker* окружење користи развојне подразумеване вредности које олакшавају локално покретање. За стварну поставку потребни су посебни налози сензора, контрола приступа темама и заштићен пренос.

MonitoringHub прослеђује подржане догађаје свим повезаним клијентима. Клијентско филтрирање по изабраној сесији није серверска изолација приступа. Ако више наставника надгледа различите испите, потребно је увести проверу права на сесију и групе за слање одговарајућих обавештења.

8. Перформансе и оптимизације

Оптерећење система зависи од броја сензора, броја видљивих уређаја и учесталости порука. У почетној спецификацији наведени су циљеви доступности од најмање 99% током сесије и кашњења до две секунде. Они су захтеви за будућу проверу, а не измерени резултати доступне верзије.

8.1. Процена обима података

За поједностављену процену нека су S број сензора, D број уређаја које сваки сензор види и f број опажања по уређају у секунди. Очекивана стопа порука је приближно $\lambda = S \cdot D \cdot f$. У стварности не мора сваки сензор да прими сваку емисију, па овај израз служи планирању синтетичког оптерећења.

Ако три сензора за 50 уређаја шаљу по једно опажање на сваких пет секунди, добија се 30 порука у секунди. За испит од два часа то је 216.000 опажања. Ово је рачунски сценарио, без тврдње да је наведена стопа остварена мерењем.

8.2. Потенцијална уска грла

Пријем једне поруке може укључити проверу *eventId*, проналажење уређаја, проверу дозволе, упис и проверу недавног аларма. Код већег оптерећења број обилазака базе може бити значајнији од саме аритметике бодовања.

PositionQueryService тренутно учитава сва опажања сесије пре издвајања временског прозора у меморији. Како сесија траје, количина учитаних података расте иако су за положај потребна само скорија мерења. Преношење временског филтера и избора последњег читавања по сензору у упит према бази представља конкретан правац оптимизације.

Извештај такође учитава опажања ради бројања. Збирне операције на серверу базе смањиле би пренос података. Поред тога, стање Калмановог филтера нема приказану политику уклањања неактивних уређаја, што треба размотрити код дуготрајног рада.

8.3. Освежавање приказа и мерење кашњења

Интервал мапе од две секунде није доказ да укупно кашњење од сензора до екрана остаје испод две секунде. Укупно време обухвата чекање на сензору, *MQTT* пренос, обраду, упис, чекање следећег упита и приказ одговора. *SignalR* обавештења имају другачији пут и треба их мерити одвојено.

За процену перформанси потребно је бележити идентификатор догађаја на границама компоненти и користити усклађене часовнике. Поред просека треба приказати медијану и високе перцентиле кашњења, број грешака, број изгубљених или поновљених порука и оптерећење базе.

9. Тестирање

Провера система обухвата алгоритме, доменска правила, *API* понашање и пролаз података кроз више компоненти. Доступни репозиторијум садржи јединичне тестове, тестове контролера са меморијском базом и сценарије симулатора. У овом документу описано је шта ти тестови проверавају; није приложен запис њиховог извршавања, нити се наводи измишљен проценат успешности.

9.1. Јединични тестови

RiskScoringServiceTests проверава већи резултат за нови недозвољени уређај, мањи резултат за дозвољени уређај и повећање при ознаци упаривања. *SessionStateTests* проверава прелаз из стања *planned* у *active*, а затим у *completed* и одбијање поновног покретања.

LocalizationServiceTests проверава да резултат за задата читавања постоји и остаје у очекиваном опсегу, као и да празан улаз враћа *null*. Назив једног теста помиње тежиште, али његов улаз садржи три сензора, па стварну покривеност треба тумачити према телу теста и извршној грани алгоритма.

KalmanPositionSmootherTests проверава постепену промену положаја уместо тренутног скока. *DeviceObservationTests* обухвата дозвољено мало одступање часовника и одбијање сувише будућег времена. Посебан тест проверава да *RssiIngestionService* прослеђује поруку пријемном процесу.

9.2. Тестови контролера и интеграциона провера

Пројекат *IntegrationTests* садржи провере пријаве са неисправним и исправним подацима. Контролер се позива непосредно, а користи се *EF InMemory* база. Проверавају се врста *HTTP* резултата, модел одговора и постојање токена за обнову.

Ове провере не покрећу прави *HTTP* сервер, *PostgreSQL*, *MQTT* посредник и *SignalR* клијент истовремено. Због тога не представљају потпуну проверу читавог распоређеног система. Посебно треба испитати миграције, ограничења *PostgreSQL* базе, паралелни пријем и прекид комуникације после уписа опажања.

9.3. Симулатор сензора

Симулатор може да се пријави преко *API*-ја, креира и покрене сесију и затим објављује *MQTT* поруке. Подржава сценарије *normal*, *high-risk*, *whitelist*, *localization*, *edge-cases*, *load* и *demo*. Команда *suite* повезује више провера у један пролаз [1].

Сценарији провере и њихов домет.

- *normal*. Улаз: Очитавања са мањим ризиком. Предмет провере: Пријем и приказ уређаја.
- *high-risk*. Улаз: Означен ризични *Bluetooth* догађај. Предмет провере: Правило и појава аларма.
- *whitelist*. Улаз: Регистрована дозвола. Предмет провере: Утицај дозволе на обраду.
- *localization*. Улаз: Очитавања три сензора. Предмет провере: Доступност процене положаја.
- *edge-cases*. Улаз: Дупликати и неважећи улази. Предмет провере: Стварно понашање валидације.

- *load*. Улаз: Подесив број и брзина порука. Предмет провере: Понашање при оптерећењу.
- *demo*. Улаз: Појављивање и кретање уређаја. Предмет провере: Демонстрација мапе.

Поруке симулатора о преласку *Wi-Fi* канала представљају дијагностички текст демонстрације. Оне не значе да рачунар физички скенира те канале. Такође, синтетички *RSSI* генерисан усклађеним моделом може показати добру локализацију без доказивања тачности у реалним радио-условима.

Следеће команде представљају поступак за поновљиву проверу у припремљеном окружењу. Њихов излаз треба сачувати уз верзију кода и конфигурацију пре уношења конкретних резултата у експериментални део рада.

```
docker compose up --build -d
docker compose --profile tools run --rm --build simulator suite
docker compose --profile tools run --rm simulator demo
dotnet test backend/DeviceDetection.slnx
```

9.4. План провере на физичким сензорима

Теренска провера треба да почне једним сензором и уређајем на познатим удаљеностима. Потребно је забележити модел плоче, фирмвер, канал или режим скенирања, оријентацију и распоред просторије. Затим се прикупља више *RSSI* узорака по положају и процењују параметри слабљења за изабрано окружење.

После тога се укључују три сензора и задају контролне тачке. За сваку тачку упоређују се стварни и процењени положај. Грешка се може рачунати као квадратни корен збира квадрата разлика координата. Извештај треба да обухвати расподелу грешке, а не само један успешан пример.

Потребно је одвојено испитати празну и заузету учионицу, уређаје у суседном простору, прекид једног сензора и уређаје који мењају идентификатор. За правила ризика треба дефинисати контролне дозвољене и недозвољене сценарије, па измерити лажна упозорења и пропуштене догађаје. До завршетка тих провера не може се навести емпиријски потврђена тачност локализације или детекције.

10. Одржавање и унапређење система

Даљи развој треба усмерити на јаснији уговор података, проверу физичког слоја и поузданост обраде. Постојећа подела на слојеве пружа основу да се такве измене уводе без потпуног преуређивања интерфејса.

10.1. Конфигурација и испорука

Подешавања везе са базом, кључева за токене, хеширања и *MQTT* посредника могу се проследити кроз променљиве окружења. Вредности за *API* и *Worker* морају бити усклађене тамо где деле исти домен, нарочито за хеширање и проверу токена.

Docker Compose омогућава поновљиво локално покретање. За одржавање је потребно забележити верзију кода, примену миграција и начин резервног копирања *PostgreSQL* података. Сам волумен чува податке између покретања контејнера, али није замена за резервну копију.

10.2. Приоритети софтверског унапређења

Први приоритет је заједничка валидација *MQTT* и *HTTP* улаза. Она треба да обухвати познат сензор, постојећу активну сесију, дозвољен тип сигнала и усаглашеност теме са *sensorId* у поруци. Тако би документовани уговор постао доследно спроведен у свим транспортима.

Други приоритет је трајни механизам испоруке догађаја после уписа у базу. Уз то је потребно увести контролу истовремених покушаја креирања сесија и аларма, као и ограничења референцијалног интегритета. На тај начин би понашање било предвидљивије при отказима и повећаном оптерећењу.

Трећи приоритет обухвата модел просторија и сензора у бази. Различите учионице треба да имају сопствене координате, димензије и параметре калибрације. Филтер положаја треба ажурирати једном по новом мерењу и ресетовати у складу са сесијом.

10.3. Хардверска и алгоритамска проширења

Фирмвер треба да реализује пријем метаподатака, временско означавање, избор канала или режима и поуздано објављивање. Његова верзија треба да буде део експерименталне евиденције. Само тако се резултат може поновити и повезати са конкретним начином мерења.

Правила ризика могу се проширити трајањем присуства, стабилношћу сигнала и просторним контекстом. Међутим, сложеније правило није аутоматски тачније. Пре његовог увођења потребан је скуп означених сценарија на коме се може оценити број оправданих и погрешних упозорења.

Додатни сензори могу обезбедити више независних читавања за процену положаја. Истовремено треба размотрити одбацивање екстремних мерења и разликовање параметара за *Wi-Fi* и *BLE*. Избор поступка треба заснивати на измереном побољшању у истој контролној поставци.

10.4. Корисничке и организационе допуне

Интерфејс се може унапредити објашњењем разлога аларма, приказом старости мерења и стања сваког сензора. Прекид комуникације треба јасно разликовати од одсуства детектованих уређаја. Корисник тако добија информацију о томе да ли је систем уопште имао услове да нешто уочи.

За више наставника потребна су права приступа по сесији и серверско раздвајање обавештења. За извештаје је корисно увести коначни снимак збирних резултата пре брисања детаљних опажања. Те измене би олакшале примену у окружењу у коме више корисника истовремено организује различите испите.

11. Закључак

У раду је представљен систем за пријем и анализу метаподатака о бежичним уређајима у контексту испитних сесија. Реализована софтверска целина повезује *MQTT* пријем, *.NET* обраду, *PostgreSQL* складиште и *Angular* интерфејс. Она омогућава управљање сесијама, праћење уређаја, формирање упозорења, приближно позиционирање и израду збирних извештаја.

Главни допринос пројекта је повезивање тих функционалности у јединствен ток који се може испитивати помоћу симулатора. Слојевита организација одваја доменска правила од начина комуникације, док заједнички улазни модел омогућава да софтверски и будући физички извори користе исту обраду.

Анализа имплементације показује и границе постојећег решења. Процена ризика заснована је на фиксним правилима, координате зависе од некалибрисаних параметара *RSSI* модела, а приказана поузданост није статистички потврђена вероватноћа. Доступни код не садржи фирмвер и теренска мерења којима би се доказала тачност детекције у стварној учионици.

Због тога систем у садашњем облику представља софтверски прототип и основу за експериментални развој. Наредни кораци су реализација сензорског слоја, калибрација, уједначавање валидације и побољшање поузданости испоруке догађаја. После тих корака могуће је оценити његову практичну вредност на основу мерљивих показатеља.

Улога система остаје подршка наставнику. Радио-опажање, процењена координата и аларм пружају повод за проверу, али сами не утврђују идентитет корисника уређаја или постојање недозвољене радње. Такав начин тумачења омогућава да техничко решење буде представљено у складу са својим стварним могућностима.

Литература

- [1] Изворни код пројекта *Device-detection---ESP32*, локални репозиторијум, стање прегледано 10. 9. 2026. Кључни извори: *backend/src/Application/Localization*, *backend/src/Application/Risk*, *backend/src/Infrastructure/Ingestion*, *backend/src/Infrastructure/Persistence*, *backend/src/Api*, *frontend/src/app*, *simulator/src/index.mjs* и *docs/spec/mqtt-contract.md*. Имплементационе тврдње у раду засноване су на овим изворима.
- [2] *Espressif Systems, ESP32 Series Datasheet*. Технички подаци за класичну ESP32 серију. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (приступљено 10. 9. 2026).
- [3] *Espressif Systems, ESP-IDF Programming Guide. RF Coexistence, ESP32*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/coexist.html> (приступљено 10. 9. 2026).
- [4] *OASIS, MQTT Version 3.1.1, OASIS Standard, 29 October 2014*. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html> (приступљено 10. 9. 2026).
- [5] *Angular, Signals. Overview*, званична документација. <https://angular.dev/guide/signals> (приступљено 10. 9. 2026).
- [6] *Microsoft, Overview of ASP.NET Core SignalR*, документација за ASP.NET Core 9. <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction?view=aspnetcore-9.0> (приступљено 10. 9. 2026).
- [7] *Microsoft, Entity Framework Core. Migrations Overview*. <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/> (приступљено 10. 9. 2026).
- [8] *PostgreSQL Global Development Group, PostgreSQL 16 Documentation. Chapter 11: Indexes*. <https://www.postgresql.org/docs/16/indexes.html> (приступљено 10. 9. 2026).
- [9] Мирко Михајловић, Генерисање и валидација дигиталних диплома у оквиру платформе „Кенгур без граница“, завршни рад, Природно-математички факултет Универзитета у Крагујевцу, јун 2026. Документ достављен као узор за структуру и начин излагања; није коришћен као извор техничких тврдњи о ESP32 пројекту.

Кратка биографија кандидата

Андрија Костовић, број индекса 51/2021.

[Година и место рођења]

[Претходно образовање и завршена средња школа]

[Студијски програм и година уписа]

[Области интересовања, релевантни пројекти и искуство]

Остали подаци за биографију нису достављени. Означена поља треба заменити стварним подацима кандидата пре предаје рада.